

Detección de Enfermedades en Plantas con CNNs

Mariana Valderrama C. · Sara López M. · Alexandra Hurtado D. · Redes Neuronales y Aprendizaje Profundo · mayo 2026

1. Descripción del Problema

Este proyecto aborda la clasificación binaria de imágenes de hojas de plantas como sanas o enfermas usando el dataset PlantVillage (Hughes & Salathe, 2015), disponible en Kaggle. El dataset contiene 20,638 imágenes RGB de hojas de tomate, papa y pimiento con 15 categorías en etiqueta binaria. La distribución es altamente desbalanceada ya que hay 15.6% sanas vs. 84.4% enfermas. La detección automatizada es crítica para reducir pérdidas agrícolas un falso negativo, una planta enferma no detectada puede propagar la infección a toda la cosecha, por lo que el Recall sobre la clase enferma es la métrica prioritaria.

Pregunta de investigación: ¿Puede una red neuronal convolucional entrenada con imágenes de hojas de plantas distinguir entre plantas sanas y enfermas, y en qué medida mejora el desempeño al incorporar transfer learning frente a una CNN entrenada desde cero?

2. Arquitectura del Modelo

Usamos 3 modelos con distintos niveles de dificultad, todas usan CrossEntropyLoss y producen 2 logits (sana / enferma):

Modelo	Enfoque	Decisiones clave de diseño
M1 — Baseline	Desde cero	4 bloques Conv (32-64-128-256), BN+ReLU+MaxPool, GAP, Dense(256)+Dropout(0.5), Dense(2)
M2 — Regularizado	Desde cero	Mismo backbone + Dense(128) extra, Dropout(0.5)+Dropout(0.3), weight decay L2=1e-4
M3 — ResNet18 TL	ImageNet	Backbone congelado Fase 1; fine-tuning layer4 Fase 2. Cabeza: Dense(256)+Dropout(0.4)+Dense(2)

El transfer learning con ResNet18 se justifica por el tamaño limitado del dataset (~12k imágenes de entrenamiento) los pesos preentrenados en ImageNet tiene detectores de características que se transfieren bien a hojas de plantas, reduciendo el overfitting sin necesidad de grandes volúmenes de datos.

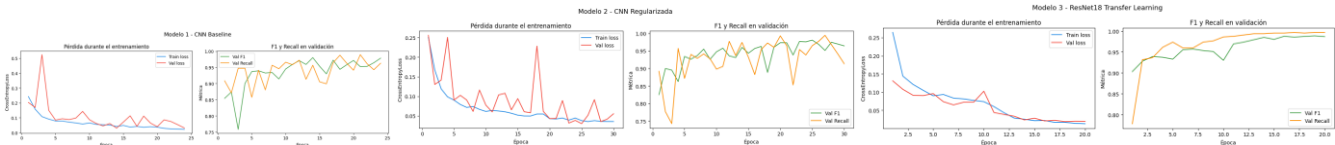
3. Procedimiento de Entrenamiento

Parámetro	M1	M2	M3 Fase 1	M3 Fase 2
Loss	CrossEntropyLoss	CrossEntropyLoss	CrossEntropyLoss	CrossEntropyLoss
Optimizador	Adam	Adam	Adam	Adam
Learning Rate	1e-3	1e-3	1e-4	1e-5
Batch Size	32	32	32	32
Epocas max.	30	30	20	20
Dropout	0.5	0.5 / 0.3	0.4	0.4
Weight Decay	—	1e-4	1e-4	1e-4

Hardware: NVIDIA Tesla T4 (15.6 GB VRAM) , Kaggle Notebooks , PyTorch 2.10.0+cu128 ,GPU CUDA.

Preprocesamiento: Todas las imágenes a 224x224 px. Normalizacion ImageNet (mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]). Augmentacion solo en train RandomHorizontalFlip, RandomVerticalFlip, RandomRotation(30), ColorJitter, RandomResizedCrop(0.8-1.0).

Curvas de entrenamiento



4. Estrategia de Validación

El dataset se dividió 60/20/20 (train/val/test) con estratificación por clase para mantener la proporción sana/enferma en cada partición (train: 12,382 , val: 4,128 , test: 4,128 imágenes). Se aplicó early stopping con paciencia=10 sobre val_F1; al detenerse se cargan automáticamente los pesos del mejor epoch. Criterio de selección: mayor val_F1 (macro) durante el entrenamiento

Métricas: F1-score (métrica principal, robusta ante desbalance), Accuracy (referencia general), Recall clase enferma (crítico minimiza falsos negativos), AUC-ROC (capacidad discriminativa).

5. Resultados

	Modelo	Accuracy	F1-score	Recall (enferma)	AUC-ROC
0	Modelo 1 — CNN Baseline	0.9886	0.9781	0.9503	0.9990
1	Modelo 2 — CNN Regularizada	0.9893	0.9798	0.9689	0.9990
2	Modelo 3 — ResNet18 TL	0.9939	0.9887	0.9984	0.9998

	Modelo	Conjunto	Accuracy	F1	Recall	AUC-ROC
0	Modelo 1 — CNN Baseline	Train	0.9894	0.9797	0.9550	0.9986
1	Modelo 1 — CNN Baseline	Validación	0.9896	0.9801	0.9566	0.9989
2	Modelo 1 — CNN Baseline	Test	0.9886	0.9781	0.9503	0.9990
3	Modelo 2 — CNN Regularizada	Train	0.9935	0.9876	0.9803	0.9992
4	Modelo 2 — CNN Regularizada	Validación	0.9896	0.9802	0.9643	0.9988
5	Modelo 2 — CNN Regularizada	Test	0.9893	0.9798	0.9689	0.9990
6	Modelo 3 — ResNet18 TL	Train	0.9982	0.9966	0.9979	1.0000
7	Modelo 3 — ResNet18 TL	Validación	0.9939	0.9887	0.9969	0.9996
8	Modelo 3 — ResNet18 TL	Test	0.9939	0.9887	0.9984	0.9998

Analisis de Overfitting / Underfitting

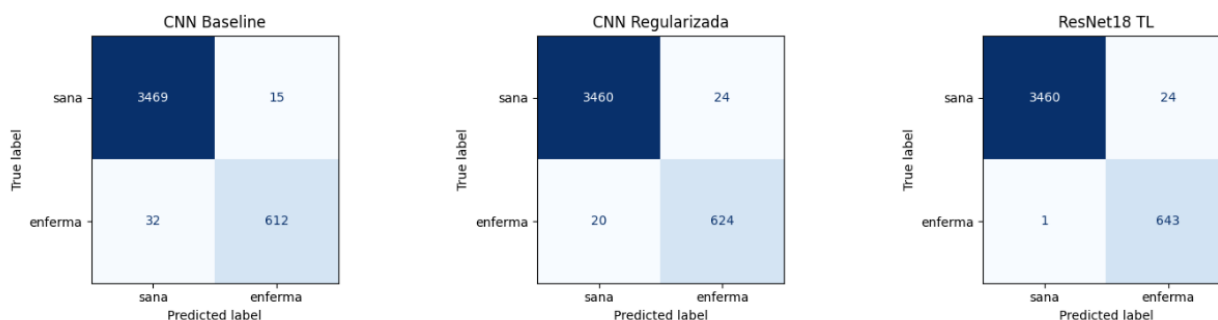
Modelo 1 (Baseline): entrenó 24 épocas (early stopping, mejor F1=0.9801). La train_loss bajo de 0.2425 a 0.0218 de forma sostenida, mientras el val_loss mostro picos en las épocas 3 y 9 , indicando overfitting moderado. El gap entre curvas se amplió a partir de la época 15, evidenciando que el modelo comienza a memorizar los datos de entrenamiento.

Modelo 2 (Regularizado): completo las 30 épocas sin activar early stopping. El val_loss fue significativamente más estable (sin picos abruptos), confirmando que el Dropout doble y el weight decay redujeron el overfitting. Métricas similares al Modelo 1, lo que indica que el cuello de botella no es el overfitting sino la capacidad de representación del backbone.

Modelo 3 (ResNet18 TL): en la Fase 2 (fine-tuning layer4), train_loss bajo de 0.0602 a 0.0129 y val_loss de 0.0439 a 0.0197, ambas en descenso paralelo y sin oscilaciones — el comportamiento más saludable de los tres modelos. Gap final de apenas 0.007, el menor de todos. Los pesos preentrenados actúan como regularización implícita. Mejores métricas: F1=0.9887, Recall=0.9969, AUC=0.9997.

Análisis de Errores

Matrices de confusión — conjunto de prueba



Falsos negativos (planta enferma clasificada como sana): el error más crítico. El Modelo 1 cometió 32, el Modelo 2 redujo a 20, y el Modelo 3 prácticamente eliminó este error con solo 1 falso negativo en todo el conjunto de prueba. Ocurren principalmente con infecciones en etapas tempranas cuyos síntomas visuales son sutiles.

Falsos positivos (planta sana clasificada como enferma): ocurren con hojas sanas que presentan patrones inusuales por iluminación, sombras o envejecimiento natural. Son menos críticos operativamente pero incrementan costos de tratamiento innecesario.

El Modelo 3 (ResNet18 TL) prácticamente elimina los falsos negativos (solo 1 en 4,128 imágenes de prueba), confirmando que el transfer learning es la estrategia más adecuada para este problema.

Referencias

- [1] Hughes, D. & Salathe, M. (2015). An open access repository of images on plant health. arXiv:1511.08060.
- [2] He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR 2016. arXiv:1512.03385.
- [3] Mohanty, S.P., Hughes, D., Salathe, M. (2016). Using Deep Learning for Image-Based Plant Disease Detection. Frontiers in Plant Science, 7, 1419.

[4] Saleem, M.H. et al. (2019). Plant Disease Detection and Classification by Deep Learning. *Plants*, 8(11), 468.